

Avaliação do Trabalho Final — MediClaw

Disciplina: Construção de APIs para Inteligência Artificial
Avaliador (simulado): Prof. Rogério | Data: 22/06/2026 | Prazo: 30/06/2026

Visão Geral do Projeto

O **MediClaw** é um sistema de apoio à decisão clínica (CDSS) que combina Django REST Framework, LLMs (OpenAI / Gemini) e RAG com ChromaDB, entregue como aplicação full-stack (backend Django + frontend Next.js). O cenário é relevante e realista: médicos interagem com um assistente de IA durante atendimentos.

1. Funcionalidade de IA ■ ATENDE (com ressalvas)

Endpoints de IA implementados (mínimo exigido: 2):

- POST /api/v1/conversations/{id}/messages/ — LLM com guardrails + RAG (modo síncrono) [conversations/views.py:103-117]
- GET /api/v1/conversations/{id}/stream/ — LLM com streaming SSE, guardrails pré/pós, RAG, captura conversacional [conversations/views.py:120-265]
- POST /api/v1/admin/knowledge/upload/ — Ingestão de documentos via embeddings (text-embedding-3-small) [rag/views.py:23-58]
- GET /api/v1/health/summary/ — Skill de agregação de saúde com análise por IA [health_logs/views.py:87-96]

Orquestrador de IA:

- generate() [orchestrator.py:183-258]: Guardrail pré-LLM → Captura → Busca RAG → LLM → Guardrail pós-LLM → Disclaimer
- generate_stream() [orchestrator.py:261-349]: variante com streaming token a token via SSE

Provedores configuráveis:

- OpenAIProvider [openai_provider.py:1-52]: stream, complete, complete_json
- GeminiProvider [gemini_provider.py:1-83]: stream, complete, complete_json

■ ■ Ressalva: ai_engine/urls.py está vazio (urlpatterns = []). As rotas estão em conversations e rag. Não é problema funcional, mas pode causar estranheza na revisão.

2. Validação de Dados ■ ATENDE

- PASSWORD_RX regex + RegisterSerializer: e-mail (unicidade), senha (≥8 chars, letra+número), aceite de termos obrigatório
- Upload: MIME whitelist (PDF, TXT, MD) + limite de 10 MB
- Mensagem de chat: CreateMessageInput (content obrigatório)
- Prompt não pode ser vazio no SSE (retorna VALIDATION_ERROR)
- Limite de mensagens: máximo 50 por conversa (MAX_MESSAGES)
- Guardrails de entrada: diagnóstico, prescrição, urgência, gibberish

3. Tratamento de Erros ■ ATENDE

- AppError(code, message, status_code, details): exceção base estruturada [exceptions.py:5-19]
- envelope_exception_handler: envolve todos os erros em {data, error, meta} [exceptions.py:31-44]
- EnvelopeJSONRenderer: contrato uniforme de resposta em todos os endpoints [renderers.py:4-12]
- LLMProviderError com HTTP 502 quando o provider de IA falha [exceptions.py:25-28]
- Erros no SSE emitidos como {"type": "error", ...} sem quebrar o stream [conversations/views.py:255-260]

- Guardrail bloqueado: resposta canned sem chamar o LLM
-

4. Logs ■ ATENDE (com ressalva menor)

- structlog: JSON em produção, colorido em desenvolvimento; nível controlado por LOG_LEVEL
- RequestIDMiddleware: X-Request-ID automático em cada request e resposta
- Latência de cada request nos logs (latency_ms) calculada por perf_counter
- UserContextMiddleware: user_id injetado nos logs após autenticação
- Log de guardrail_blocked com fase (input/output) e razão [orchestrator.py:192-200]
- Log de stream_error dentro do gerador SSE [conversations/views.py:247-251]

■ Ressalva — Audit Trail é stub: record() é um stub com pass. Os eventos (LOGIN, USER_REGISTERED, GUARDRAIL_BLOCKED, MESSAGE_SENT, KB_UPLOAD) não são persistidos em banco. O README reconhece isso como limitação (E7 pendente).

5. Segurança ■ ATENDE

- JWT Bearer: access 30 min, refresh 1 dia [settings.py:130-138]
- Multi-tenancy: doctor=request.user em todos os querysets de pacientes
- IsAdminRole: verifica user.role == "ADMIN" para endpoints administrativos
- Rate limiting: Anônimo 30/min · Usuário 60/min · Chat 10/min
- CORS: origens explícitas via CORS_ALLOWED_ORIGINS (não *)
- Validação de senha: letra + dígito + mínimo 8 chars
- Guardrails éticos: bloqueio de diagnóstico, prescrição, urgência simulada

■ Ressalvas documentadas: Token JWT via query string no SSE (?token=...) — limitação do EventSource nativo. JWT em localStorage — risco teórico de XSS. Ambas com mitigações futuras planejadas.

6. Versionamento ■ ATENDE

- Todas as rotas usam prefixo /api/v1/... [urls.py:31-37]
 - CI/CD: stages de lint (Black + pre-commit) e test (pytest com PostgreSQL 16 real) [.gitlab-ci.yml:1-38]
 - Pre-commit hooks: Black para formatação automática
 - Repositório git ativo (branch main)
-

7. Documentação ■ ATENDE (destaque positivo)

- Swagger interativo: GET /swagger/ via drf-yasg
 - ReDoc: GET /redoc/
 - Healthcheck: GET /health/ retorna status de DB e vector store
 - README acadêmico completo: resumo executivo, arquitetura, stack, funcionalidades, limitações
 - PRD e decisões arquiteturais (ADRs) em specs/PRD.md e specs/ARCHITECTURE.md
 - Instruções de instalação passo a passo para backend e frontend
-

8. Qualidade do Código ■ DESTAQUE POSITIVO

- 137 testes automatizados no backend com pytest + pytest-django
 - 22 testes de guardrails: urgência, diagnóstico, prescrição, gibberish, saída do LLM
 - ~40 testes de frontend com Vitest + Testing Library
 - Provider pattern: OpenAI ↔ Gemini intercambiáveis via variável de ambiente LLM_PROVIDER
 - Envelope API uniforme {data, error, meta}
 - Monolito modular (apps por domínio), service layer, formatação com Black + pre-commit
-

9. Reprodutibilidade ■ ATENDE

- Passo a passo completo para backend e frontend no README
- Todas as variáveis de ambiente documentadas em .env.example
- CI roda pytest com PostgreSQL 16 real (não SQLite) [.gitlab-ci.yml:28-37]
- Suporte a DevContainer / GitHub Codespaces [.devcontainer/]
- requirements.txt com hashes SHA256 para instalação determinística

Resumo da Avaliação por Critério

Critério	Status	Nota	Observação
Funcionalidade de IA (≥2 endpoints)	■	Excelente	4 endpoints IA + streaming SSE
Dados válidos e inválidos	■	Excelente	Guardrails + serializers + testes
Executa em outro computador	■	Bom	README + .env.example
Validação de dados	■	Excelente	Múltiplas camadas
Tratamento de erros	■	Excelente	Envelope padronizado
Logs	■	Bom	structlog OK; audit não persiste
Segurança	■	Bom	JWT + rate limiting + CORS
Versionamento	■	Excelente	v1 nas URLs + CI
Qualidade e documentação	■	Excelente	137 testes + Swagger + README
Repositório público	■■	Verificar	Garantir acesso no GitHub
Apresentação (vídeo)	■	Pendente	Entregável não concluído

Pontos Fortes (para destacar na apresentação)

- 1. Pipeline de IA completo e diferenciado: guardrails pré e pós-LLM, RAG com ChromaDB, captura conversacional de dados clínicos, streaming SSE em tempo real.
- 2. 137 testes automatizados com cobertura de guardrails, orquestrador, skills, RAG e autenticação.
- 3. Documentação acadêmica excepcional no README — evolução do projeto, arquitetura, decisões técnicas e limitações.
- 4. Provedores LLM intercambiáveis (OpenAI ↔ Gemini via variável de ambiente) sem mudança de código.
- 5. Swagger/ReDoc integrado e acessível.

Pontos a Corrigir ou Completar Antes da Entrega

Crítico:

- [] Garantir repositório público no GitHub (não apenas GitLab). O professor exige link do GitHub.
- [] Gravar vídeo de apresentação de até 10 minutos (demonstração funcional + características técnicas).

Importante (impacta nota):

- [] Implementar audit trail funcional — substituir o pass em audit/services/log.py por persistência real.
- [] Adicionar ao README seção de testes documentando como executar (pytest tests/ -v e npm test).

Recomendado:

- [] Criar pelo menos 1 cenário de teste de dados inválidos demonstrável durante a apresentação.
- [] Adicionar link do Swagger no README raiz.
- [] Verificar se NEXT_PUBLIC_API_URL está documentado no react-painel/.example.env.local.

Observações Finais

O projeto demonstra **domínio avançado** dos conceitos da disciplina. Vai além do mínimo exigido em todas as dimensões técnicas: entrega streaming SSE, RAG, múltiplos provedores de LLM, guardrails éticos, suite de testes robusta e documentação de nível profissional.

Os dois pontos que podem comprometer a nota são **operacionais**, não técnicos:

- 1. O repositório precisa estar acessível no GitHub conforme exigido.
- 2. O vídeo de apresentação é entregável obrigatório e ainda não foi concluído.

O audit trail stub é a única lacuna técnica relevante. Os demais itens pendentes (E7: testes E2E, hardening de produção, cookies httpOnly) estão bem documentados como trabalhos futuros e não penalizam a avaliação.

Prazo restante: 8 dias (entrega: 30/06/2026).